DOCUMENTACIÓN DE DISEÑO DEL SOFTWARE

LAVA RÁPIDO VEHICULAR

Interfaces gráficas, modelo arquitectónico, patrones y modelo de datos

INTEGRANTES

José Stiven Perdomo Martínez

Santiago Gordo Pérez

SERVICIO NACIONAL DE APRENDIZAJE – SENA

Tecnólogo en Análisis y Desarrollo de Software

Ficha 3145555

2026

Contenido

1. Resumen e introducción

2. Objetivos, alcance y principios

3. Diseño de interfaces gráficas

4. Modelo arquitectónico

5. Patrones aplicados y evidencias

6. Modelo conceptual de datos

7. Modelo lógico relacional

8. Modelo físico PostgreSQL

9. Trazabilidad y criterios de calidad

10. Limitaciones, riesgos y conclusiones

11. Referencias

Corte documental: 4 de noviembre de 2026. Las capturas ilustran interfaces ejecutadas y el inventario técnico caracteriza los archivos de los repositorios revisados. Las versiones desplegadas y el estado efectivo de una base de datos no se certifican mediante este documento.

1. Resumen e introducción

Lava Rápido Vehicular es una solución para la atención de servicios de autolavado mediante aplicación móvil para clientes y operadores, panel administrativo web y API REST con persistencia centralizada. Este documento detalla las decisiones de diseño y reúne evidencia visual de la aplicación, modelos de arquitectura, patrones empleados y modelos de datos conceptual, lógico y físico.

El diseño es cliente-servidor y utiliza un backend monolítico modular Spring Boot, React Native/Expo y React/Vite en la presentación, y PostgreSQL con Liquibase en persistencia. La solución articula reservas, pagos con Wompi, asignaciones, notificaciones y calificaciones, con autorización por rol y propiedad de recursos.

El documento se apoya en la especificación de requisitos de Lava Rápido Vehicular, la propuesta técnica, el documento de diseño previo, las capturas facilitadas por el equipo y el inventario de archivos y clases de los repositorios. Se describe el estado disponible, sin atribuir resultados de ejecución a código únicamente identificado.

2. Objetivos, alcance y principios

2.1 Objetivo general

Documentar la estructura, interfaces, patrones y persistencia de Lava Rápido Vehicular, vinculando las decisiones de diseño con los requisitos funcionales y no funcionales del proyecto formativo.

2.2 Objetivos específicos

- Representar y analizar interfaces reales del cliente móvil y del panel administrativo.
- Presentar modelos de arquitectura de contexto, capas y componentes.
- Identificar patrones implementados y su ubicación verificable en los repositorios.
- Construir modelos de datos conceptual, lógico y físico a partir de las migraciones SQL declaradas.
- Explicitar restricciones, riesgos y límites de verificación del diseño.

2.3 Alcance y principios

Se incluyen identidad, catálogo, vehículos y marcas, reservas, pagos, operadores, asignaciones, notificaciones, calificaciones y preferencias. Fuera de alcance: GPS en tiempo real de operadores, optimización de rutas, inventario físico y contabilidad detallada. La ubicación de un punto de atención en mapa no representa seguimiento GPS.

Principios rectores: separación por rol; validación de permisos en backend; estados de reserva, pago y asignación diferenciados; horario 06:00 a 20:00; precio y duración pactados inmutables para la reserva; control de solapamientos por vehículo; información de pago confirmada mediante proveedor.

3. Diseño de interfaces gráficas

Se incorporan doce capturas reales proporcionadas por el equipo: seis del panel web administrativo y seis de la aplicación móvil. Se han ocultado los campos potencialmente personales en las imágenes, incluyendo nombres, correos, teléfonos y placas. Las figuras sirven como evidencia visual de pantallas, pero no como prueba completa de todos sus flujos de interacción.

3.1 Diseño visual y navegación

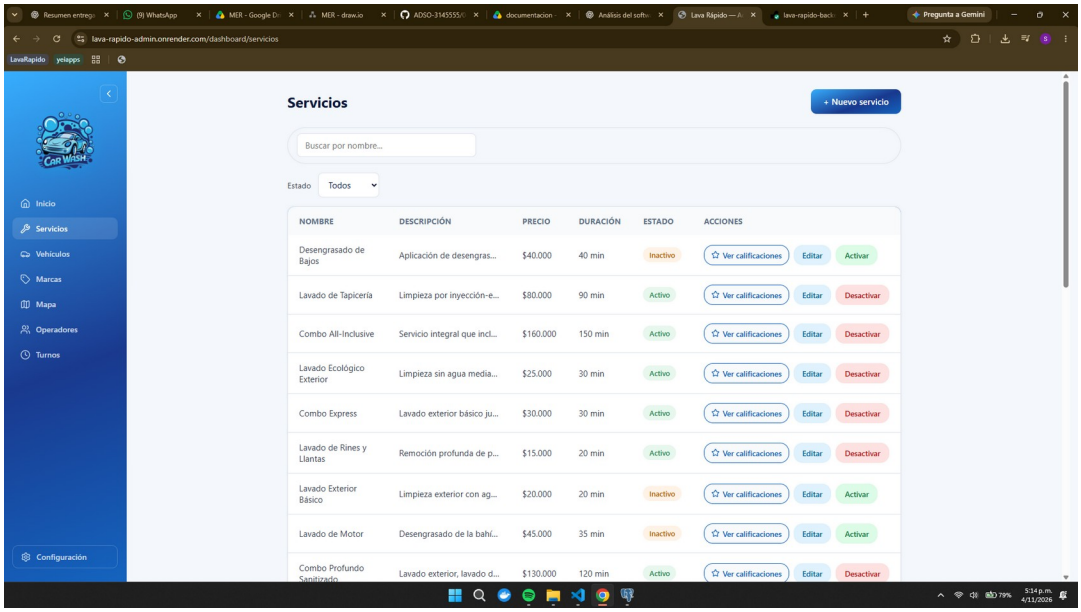
El panel presenta barra lateral azul para módulos, tablas de gestión, filtros, estados y acciones contextuales. La app móvil emplea navegación inferior para Inicio, Reservas, Mapa, Avisos y Perfil, tarjetas de servicios, filtros y estados de lectura. El código identifica ThemeContext en móvil, recursos i18n es/en/pt/fr y variables de estilo compartidas en web.

Tabla 1. Inventario y función de interfaces aportadas

Canal	Pantalla	Función principal
Web	Gestión de servicios	Mantener catálogo y disponibilidad
Web	Gestión de marcas	Consultar y editar marcas
Web	Ubicaciones de atención	Consultar puntos de atención
Web	Gestión de operadores	Consultar y administrar operadores
Web	Turnos y reservas	Supervisar y registrar reservas
Web	Panel principal	Visualizar indicadores operativos
Móvil	Inicio del cliente	Acceder al catálogo desde inicio
Móvil	Catálogo de servicios	Buscar servicios y filtrar disponibilidad
Móvil	Seguimiento de reservas	Consultar reservas por estado
Móvil	Ubicaciones móviles	Ver punto de atención en mapa
Móvil	Notificaciones	Consultar y marcar avisos
Móvil	Perfil del cliente	Consultar información de cuenta

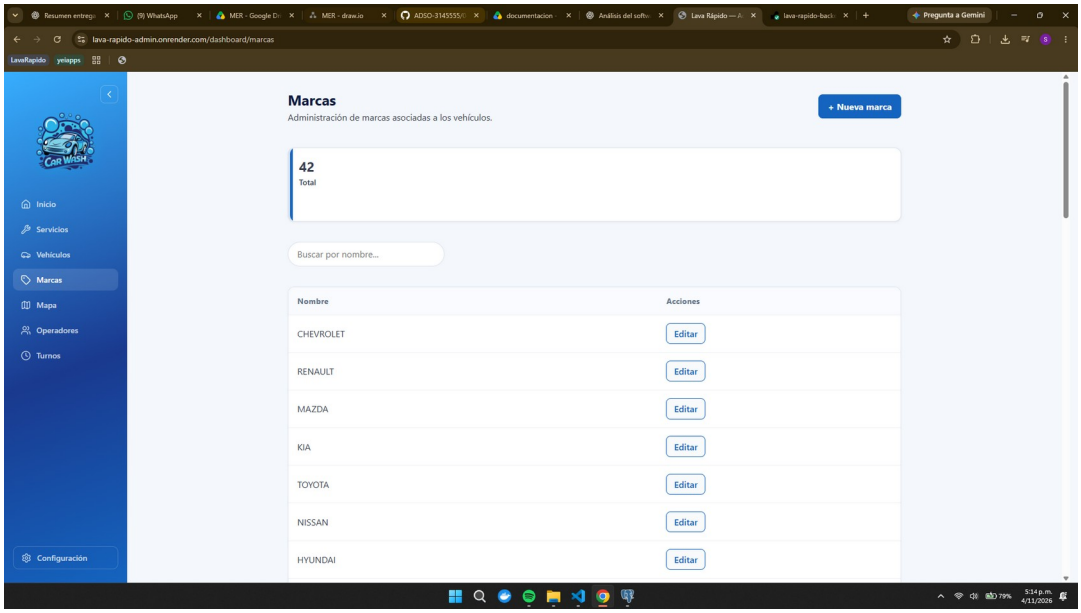
3.2 Evidencia visual del panel web (1/3)

Figura 1. Gestión de servicios (panel administrativo).



Nota. Captura facilitada por el equipo; datos personales ocultados cuando corresponde.

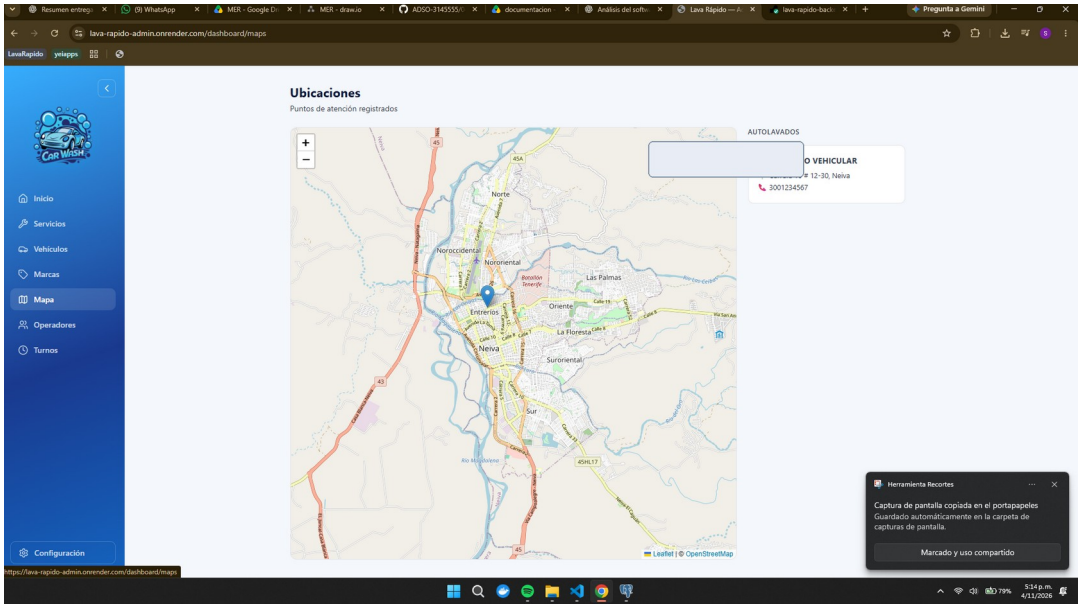
Figura 2. Gestión de marcas (panel administrativo).



Nota. Captura facilitada por el equipo; datos personales ocultados cuando corresponde.

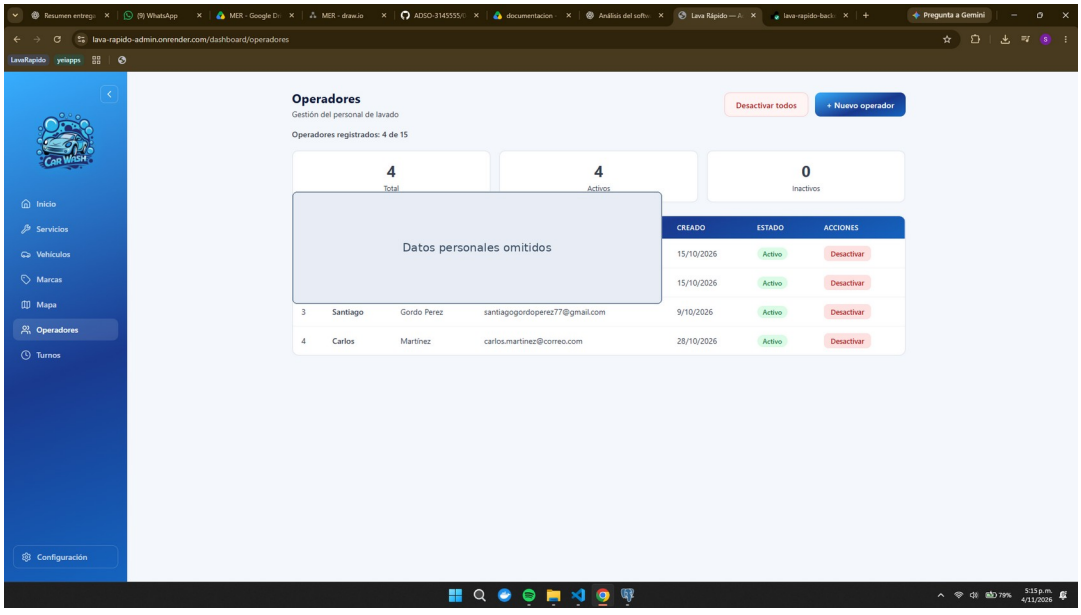
3.2 Evidencia visual del panel web (2/3)

Figura 3. Ubicaciones de atención (panel administrativo).



Nota. Captura facilitada por el equipo; datos personales ocultados cuando corresponde.

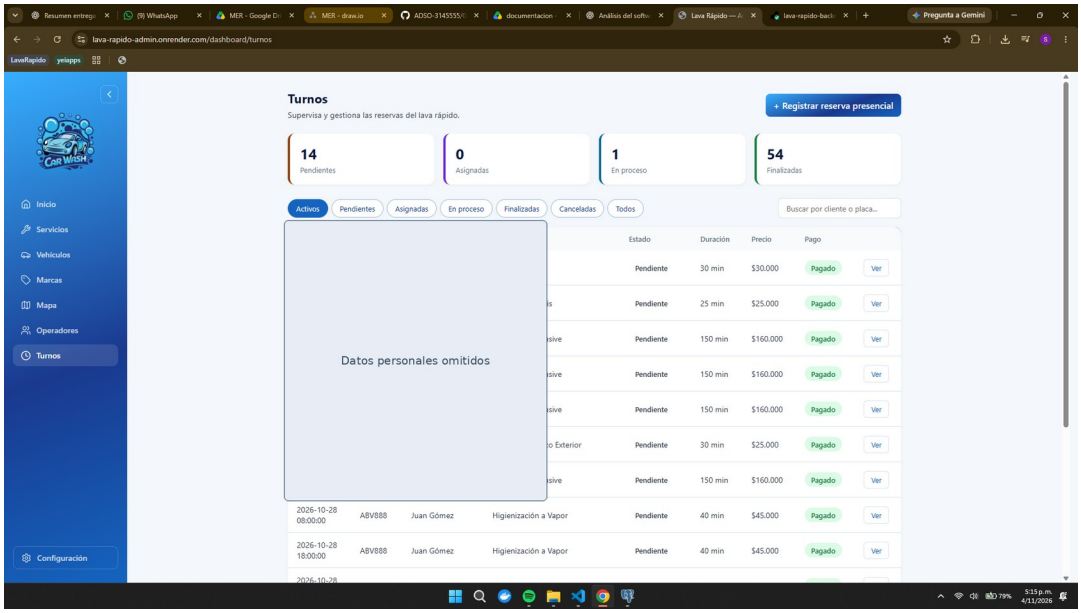
Figura 4. Gestión de operadores (panel administrativo).



Nota. Captura facilitada por el equipo; datos personales ocultados cuando corresponde.

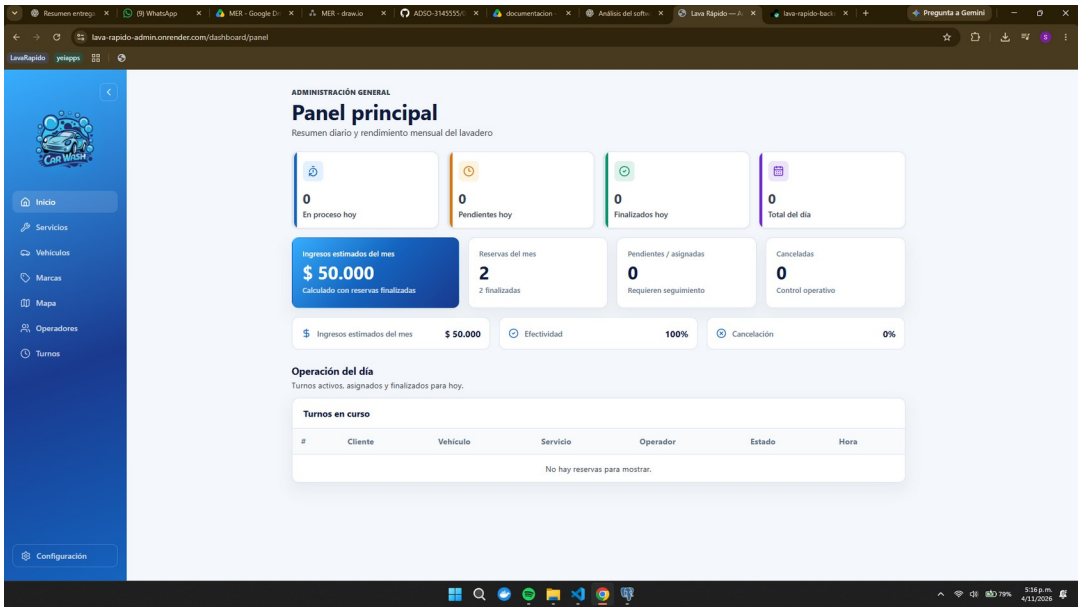
3.2 Evidencia visual del panel web (3/3)

Figura 5. Turnos y reservas (panel administrativo).



Nota. Captura facilitada por el equipo; datos personales ocultados cuando corresponde.

Figura 6. Panel principal (panel administrativo).



Nota. Captura facilitada por el equipo; datos personales ocultados cuando corresponde.

3.3 Evidencia visual de la aplicación móvil (1/2)

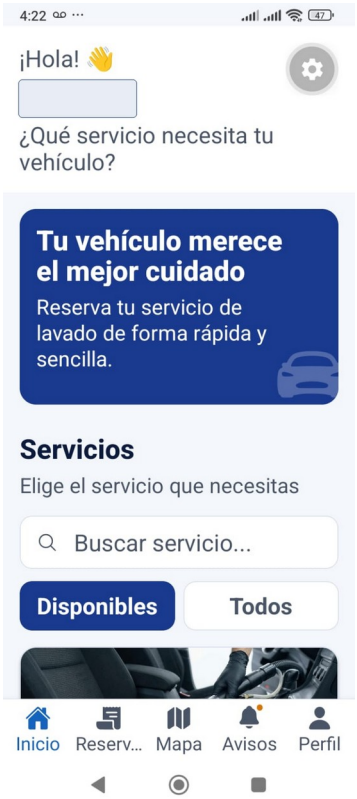


Figura 7. Inicio del cliente.



Figura 8. Catálogo de servicios.

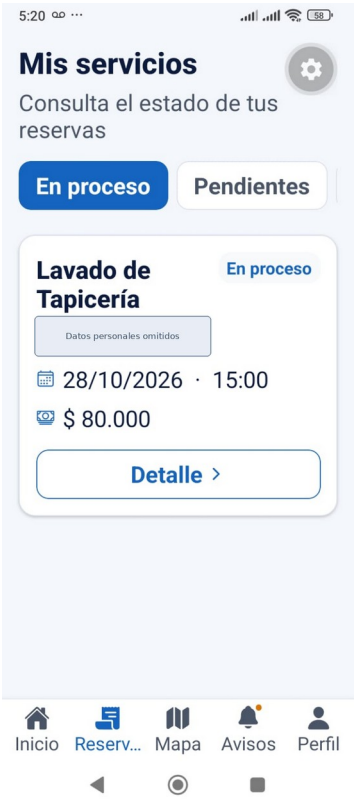


Figura 9. Seguimiento de reservas.

Nota. Capturas de Android facilitadas por el equipo; datos identificadores omitidos. Se conserva el diseño visual restante.

3.3 Evidencia visual de la aplicación móvil (2/2)

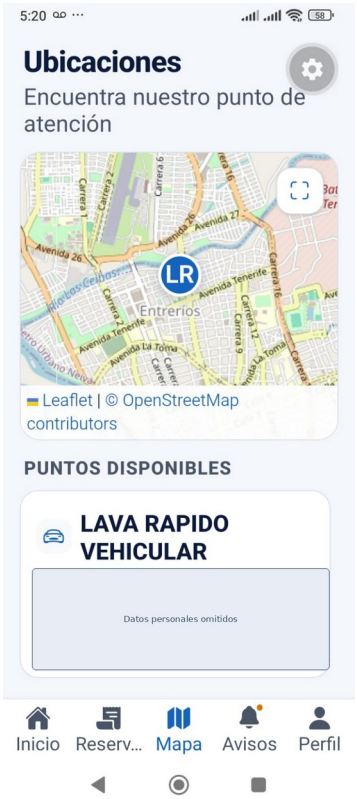


Figura 10. Ubicaciones móviles.

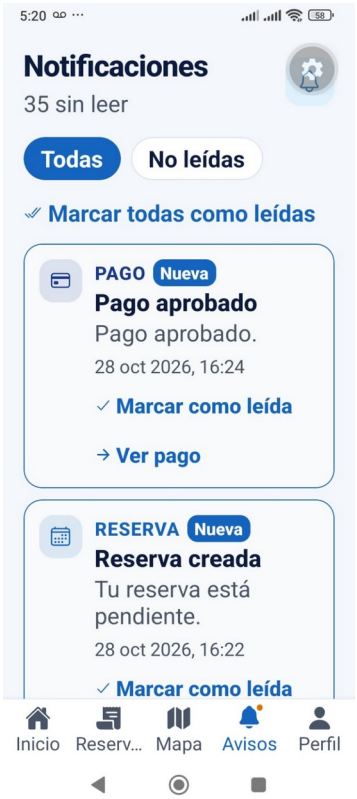


Figura 11. Notificaciones.



Figura 12. Perfil del cliente.

Nota. Capturas de Android facilitadas por el equipo; datos identificadores omitidos. Se conserva el diseño visual restante.

3.4 Rutas y cobertura de interfaz

En web se identifican /dashboard/panel, /dashboard/servicios, /dashboard/vehiculos, /dashboard/marcas, /dashboard/maps, /dashboard/operadores, /dashboard/turnos, /dashboard/perfil y /dashboard/configuracion. El cobro se inicia desde el contexto de reserva y /pagos/resultado presenta el resultado; no se identificó una página general independiente de CRUD de pagos. /dashboard/notificaciones redirige al panel, no constituye una bandeja administrativa.

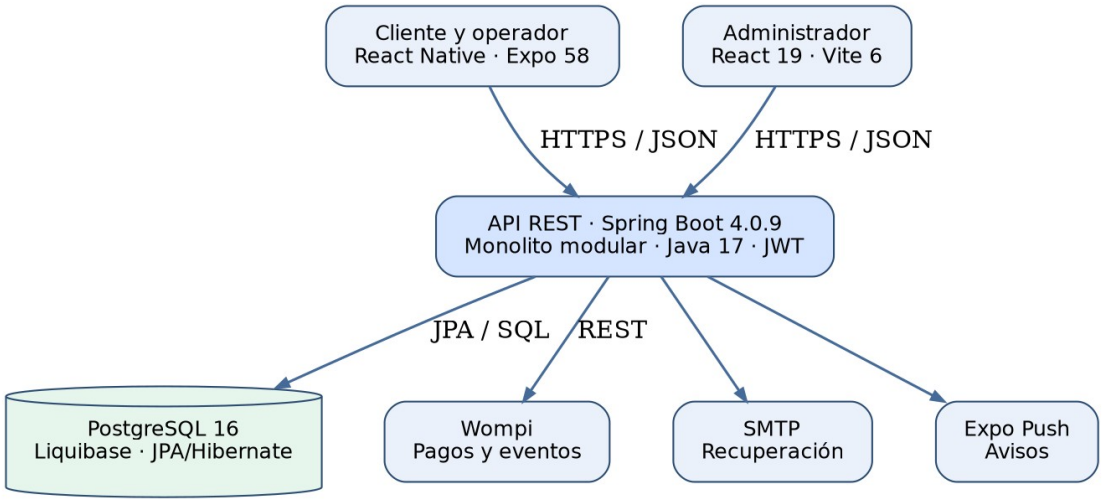
En móvil, HomeScreen y ServiceDetailScreen atienden catálogo y detalle; MyServicesScreen y MyReservationsScreen se relacionan con el seguimiento; NotificationsScreen, ProfileScreen y navegación de tabs materializan avisos y perfil. Las pantallas de cliente y operador se separan con AppNavigator, TabNavigator y OperatorNavigator.

Las capturas no abarcan registro, reserva, widget Wompi, cancelación, calificación ni navegación del operador. Esas pantallas y flujos deben documentarse mediante evidencia adicional si se requiere una muestra visual exhaustiva.

4. Modelo arquitectónico

Los clientes React Native y React consumen una API Spring Boot mediante HTTP(S) y contratos JSON. El backend único se organiza modularmente por dominios y usa Spring Security para autenticación/autorización, servicios de negocio y repositorios JPA. PostgreSQL administra la información y Liquibase controla el esquema SQL declarado.

Figura 13. Arquitectura lógica de la solución.



Nota. Modelo del sistema; las conexiones externas representan integraciones declaradas, no una prueba de operación en vivo.

4.1 Arquitectura de componentes y capas

Figura 14. Estructura de componentes del backend.



Las capas separan controladores REST, servicios de dominio, repositorios y entidades JPA. Las validaciones de autorización aplican transversalmente. Los módulos identificados incluyen auth/users, servicios, marcas, vehículos, reservas, operadores, asignaciones, pagos, calificaciones, notificaciones, auditoría y errores.

4.2 Integraciones y límites de confianza

Wompi procesa pagos en un flujo donde la redirección o callback del frontend no basta para aprobar; el backend verifica firma o consulta de transacción y contrasta referencia, monto, moneda y ambiente. SMTP apoya recuperación y Expo Push puede entregar avisos según configuración. El frontend no dispone de acceso directo a PostgreSQL ni debe recibir secretos privados de proveedores.

La configuración revisada declara Java 17/Spring Boot 4.0.9, Expo ~58.0.20/React Native 0.88, React 19/Vite 6 y PostgreSQL 16. La presencia de Docker, EAS y referencias de Render no demuestra por sí sola la versión desplegada.

5. Patrones aplicados y evidencias

Se distinguen patrones arquitectónicos, de aplicación y prácticas de integración; no se atribuyen patrones GoF sin estructuras concretas que los demuestren.

Tabla 2. Patrones, ubicación y propósito

Patrón o práctica	Ubicación real	Aplicación y ventaja
Arquitectura por capas	backend: controller / service / repository / entity	Separación entre HTTP, negocio y persistencia
Controller–Service–Repository	ReservaController → ReservaService → ReservaRepository	Orquestación de la reserva con responsabilidades delimitadas
Repository	Interfaces Spring Data; ReservaRepository, PagoRepository	Acceso a datos abstraído de las reglas de negocio
DTO	ReservaRequestDTO, ReservaResponseDTO y DTO de pagos	Contratos de entrada/salida separados de entidades
Inyección de dependencias	@Service, @Component y constructores de componentes Spring	Dependencias administradas por el framework
Adaptación de integración	WompiTransactionClient y WompiTransactionClientImpl	Encapsula HTTP externo y errores de proveedor detrás de interfaz
Manejo central de excepciones	GlobalExceptionHandler	Mapeo uniforme de errores de dominio a

		HTTP
Componentes y hooks React	Pantallas por feature; useWompiWidget	Reutilización de interfaz y encapsulación de estado
Estado compartido	authStore.ts (Zustand), ThemeContext	Sesión y apariencia sin pasar props manualmente por toda la jerarquía
Rutas protegidas	RequireAdmin.tsx y Spring Security	UI filtra navegación; backend verifica permisos

5.1 Ejemplos de aplicación

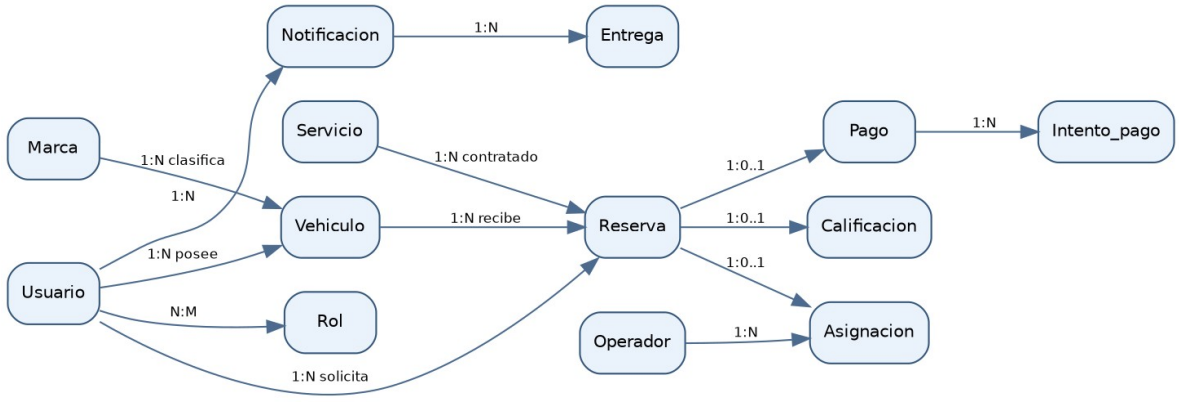
ReservaController procesa POST /api/reservas y delega la operación en ReservaService; este valida vehículo, servicio, horario, solapamiento y persistencia mediante ReservaRepository. PagoService usa la abstracción WompiTransactionClient para consultar transacciones; no se detectó selección dinámica de múltiples estrategias, por lo que no se califica como Strategy. El panel utiliza RequireAdmin para confirmar rol con el backend y authStore para estado local de sesión.

No se encontró evidencia suficiente para afirmar implementaciones explícitas de Factory, Singleton GoF ni Redux Toolkit en el frontend móvil actual. La inyección de dependencias no debe confundirse con un Singleton implementado manualmente.

6. Modelo conceptual de datos

El modelo conceptual describe los objetos del negocio y sus relaciones sin imponer estructuras SQL. Las cardinalidades se derivan de asociaciones y restricciones declaradas; la disponibilidad real de una instancia no fue comprobada.

Figura 15. Modelo conceptual de entidades y relaciones de negocio.

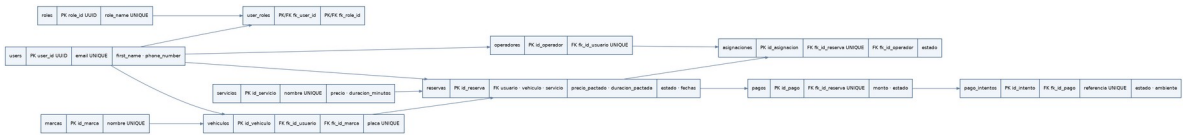


Usuario posee vehículos y solicita reservas. Cada reserva vincula un vehículo y un servicio y puede asociarse con un pago, una asignación y una calificación. El pago admite varios intentos; una asignación vincula la reserva con un operador. Las notificaciones comunican eventos a usuarios. Los roles de usuario forman una relación de muchos a muchos mediante una entidad asociativa.

7. Modelo lógico relacional

El modelo lógico traduce entidades a tablas, atributos y relaciones PK/FK sin fijar detalles de despliegue. Se presenta un núcleo operativo y se enumeran las tablas satélite en el modelo físico. La tabla user_roles resuelve la relación N:M entre users y roles.

Figura 16. Modelo lógico principal con claves y relaciones.



En el modelo lógico, pagos.fk_id_reserva, asignaciones.fk_id_reserva y calificaciones.fk_id_reserva son únicos. Esta restricción produce relaciones 1 a 0..1 con reservas. pago_intentos registra múltiples intentos ligados a un pago. El snapshot en reservas permite conservar precio y duración pactados.

8. Modelo físico PostgreSQL

El modelo físico se basa en los SQL incluidos en el changelog de Liquibase. Declara 18 tablas y 24 changeSets: 15 de DDL y 9 de DML. Esta cifra corresponde a archivos, no a una base instalada. Los datos de inicialización no se consideran evidencia de producción.

Figura 17. Organización física de las 18 tablas por dominio.

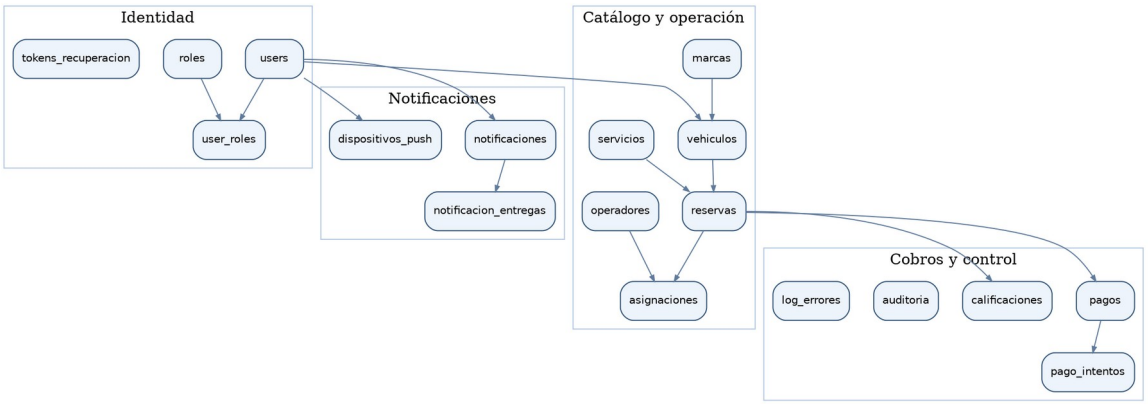


Tabla 3. Diccionario físico resumido; PK, FK, UNIQUE y tipos de columnas clave

Tabla	Columnas y tipos representativos del DDL
roles	role_id UUID PK; role_name VARCHAR(20) UNIQUE; description VARCHAR(255)
users	user_id UUID PK; email VARCHAR(100) UNIQUE; first_name VARCHAR(50); phone_number VARCHAR(10); document_number VARCHAR(12)
user_roles	fk_user_id UUID PK/FK; fk_role_id UUID PK/FK; status BOOLEAN; revoked_at TIMESTAMP
tokens_recuperacion	id_token UUID PK; fk_id_usuario UUID FK; token_hash

	VARCHAR(64) UNIQUE; expiracion_at TIMESTAMP
servicios	id_servicio UUID PK; nombre VARCHAR(100) UNIQUE; precio NUMERIC(10,0); duracion_minutos INT
marcas	id_marca UUID PK; nombre VARCHAR(30) UNIQUE; fk_id_usuario_solicitante UUID FK
vehiculos	id_vehiculo UUID PK; fk_id_usuario UUID FK; fk_id_marca UUID FK; placa VARCHAR(7) UNIQUE
reservas	id_reserva UUID PK; FKs usuario/vehículo/servicio; fecha_reserva DATE; hora_reserva TIME; precio_pactado NUMERIC(12,0); duracion_minutos_pactada INTEGER; estado VARCHAR(20)
pagos	id_pago UUID PK; fk_id_reserva UUID FK UNIQUE; monto NUMERIC(12,0); estado VARCHAR(15)
calificaciones	id_calificacion UUID PK; fk_id_reserva UUID FK UNIQUE; fk_id_usuario UUID FK; puntuacion INT
auditoria	id_auditoria UUID PK; fk_id_usuario UUID FK opcional; accion VARCHAR(30); descripcion VARCHAR(500)
log_errores	id_error UUID PK; fk_id_usuario UUID FK opcional; tipo_error VARCHAR(30); resuelto BOOLEAN
operadores	id_operador UUID PK; fk_id_usuario UUID FK UNIQUE; estado BOOLEAN
asignaciones	id_asignacion UUID PK; fk_id_reserva UUID FK UNIQUE; fk_id_operador UUID FK; estado VARCHAR(15)
pago_intentos	id_intento UUID PK; fk_id_pago UUID FK; referencia VARCHAR(100) UNIQUE; wompi_environment VARCHAR(10); estado VARCHAR(24)
notificaciones	id_notificacion UUID PK; fk_id_usuario UUID FK; clave_evento VARCHAR(180); fk_id_reserva UUID FK opcional; leida_at TIMESTAMP
dispositivos_push	id_dispositivo UUID PK; fk_id_usuario UUID FK; token VARCHAR(255) UNIQUE; activo BOOLEAN
notificacion_entregas	id_entrega UUID PK; fk_id_notificacion UUID FK; canal VARCHAR(10); estado VARCHAR(12); intentos INTEGER

El diccionario presenta columnas principales, no una transcripción exhaustiva de todas las columnas o restricciones. Para reproducir completamente la estructura se deben utilizar los scripts SQL versionados de Liquibase del repositorio de base de datos.

8.1 Integridad del esquema

- Reservas: restricción de exclusión GiST por vehículo e intervalo de tiempo, utilizando btree_gist; impide solapamientos según la condición declarada.
- Snapshot: reservas.precio_pactado y duracion_minutos_pactada conservan valores históricos y tienen validaciones de positividad.
- Cobros: un pago por reserva; referencias de intentos únicas y controles parciales para transacción por ambiente e intento pendiente por pago.
- Asignación y calificación: restricciones UNIQUE por reserva.
- Notificaciones: unicidad por usuario y clave de evento; índices parciales para entregas de correo y push.
- Roles y estados: restricciones CHECK para dominios y consistencia; enums Java mapeados a cadenas, sin tipos ENUM PostgreSQL declarados.

8.2 Distinción entre JPA y SQL

Varias relaciones de notificaciones se modelan en Java mediante identificadores UUID simples aunque tienen FK en SQL. Los índices parciales y las restricciones de exclusión se expresan en SQL, no completamente en anotaciones JPA. La correspondencia con una base activa debe validarse con ejecución controlada de Liquibase y comprobaciones del esquema, sin presumir que ya están aplicadas.

9. Trazabilidad y criterios de calidad

Tabla 4. Trazabilidad de diseño frente a requisitos del SRS

RF	Elementos de diseño que lo soportan
RF1	Pantallas de autenticación, UserController, SecurityConfig, users/roles/tokens_recuperacion
RF2–RF4	Catálogo móvil, panel Servicios, ServicioService y tabla servicios
RF5	CalificacionService, calificaciones y reserva FINALIZADA
RF6–RF7	Flujo Wompi, PagoService, pagos y pago_intentos
RF8 / RF11	Mis servicios, Turnos, ReservaService y exclusión de intervalos
RF9 / RF10	Perfil, vehículos, ThemeContext e internacionalización
RF12	Panel de operadores, navegación de operador y asignaciones
RF13	Notificaciones móviles y tablas notificaciones/dispositivos_push/notificacion_entregas

La aceptación de diseño requiere coherencia entre UI y endpoints, seguridad por rol y propietario, control de errores y consistencia entre modelo JPA y SQL. Una captura o declaración de componente no demuestra por sí sola un flujo E2E exitoso.

10. Limitaciones, riesgos y conclusiones

10.1 Limitaciones verificables

- Las capturas corresponden a pantallas facilitadas por el equipo; no se ejecutó una navegación automatizada de toda la aplicación durante la elaboración.
- El inventario SQL corresponde al esquema declarado, sin acceso de lectura a la base instalada ni comprobación de DATABASECHANGELOG.
- Las integraciones Wompi, SMTP, Expo Push y la versión efectiva de los servicios en Render no se probaron en esta elaboración.
- Faltan capturas de registro, reserva/pago, y navegación del operador para una galería exhaustiva.

10.2 Riesgos y acciones

Riesgo	Control o validación recomendada
Diferencia entre migración y base activa	Inspeccionar DATABASECHANGELOG y validar esquema/índices en ambiente autorizado
Estados operativos o pagos inconsistentes	Pruebas integradas de transición, idempotencia y conciliación

	Wompi
Acceso no autorizado	Pruebas API por rol, propietario y token caducado
Pantallas desactualizadas	Comparar capturas de fecha conocida con compilación y commits publicados
Datos personales en evidencias	Anonimización previa a la entrega y revisión de metadatos

10.3 Conclusiones

El sistema presenta un diseño coherente con una arquitectura cliente-servidor y monolito modular. El backend separa controlador, servicio y repositorio y delega la consistencia de datos en validaciones de negocio, transacciones y restricciones PostgreSQL. El frontend móvil y el panel administrativo ofrecen experiencias por rol con servicios HTTP comunes.

Los modelos conceptual, lógico y físico permiten explicar cómo usuarios, vehículos, reservas, servicios, operadores, pagos y notificaciones se articulan. Se distinguen adecuadamente los componentes identificados en código, las capturas reales y la verificación pendiente del entorno instalado, lo que evita atribuir al sistema capacidades no comprobadas.

11. Referencias

Lava Rápido Vehicular. (2026). Especificación de requisitos de software [Documento interno del proyecto]. Servicio Nacional de Aprendizaje.

Lava Rápido Vehicular. (2026). Análisis del software: Vistas dinámicas [Documento interno del proyecto]. Servicio Nacional de Aprendizaje.

Lava Rápido Vehicular. (2026). Propuesta técnica del software [Documento interno del proyecto]. Servicio Nacional de Aprendizaje.

Lava Rápido Vehicular. (2026). Repositorios del frontend, backend y base de datos: Código y migraciones SQL [Material técnico del proyecto, corte al 4 de noviembre de 2026].

PostgreSQL Global Development Group. (s. f.). PostgreSQL documentation. https://www.postgresql.org/docs/

Liquibase. (s. f.). Liquibase documentation. https://docs.liquibase.com/

Oracle. (s. f.). Java documentation. https://docs.oracle.com/en/java/

Meta Open Source. (s. f.). React documentation. https://react.dev/

Spring. (s. f.). Spring Boot reference documentation. https://docs.spring.io/spring-boot/